Вариант №27

Параметры варианта

Ось	Значение	Используется
Источник данных	sensor.community, датчики качества воздуха	ЛР1–ЛР9
Фильтр потока	страны PL, CZ и AT	ЛР1–ЛР9
Способ описания схемы	Avro	ЛР2, ЛР6, ЛР8
Объектное хранилище	Garage	ЛР3–ЛР9
Движок запросов	Trino	ЛР4, ЛР5, ЛР7, ЛР9
Инструмент визуализации	Apache Superset	ЛР5, ЛР8, ЛР9
Брокер	Redpanda	ЛР6, ЛР8
Оркестратор	Apache Airflow	ЛР7
Потоковый движок	Apache Flink	ЛР8

Таблица 1.1: Параметры варианта.

Вариант выдаётся на семестр; номер меняется только до защиты работы 1 (раздел «Стенд курса» методических указаний). Работа, выполненная на других продуктах или на данных другого источника, не принимается. Замена источника, ставшего недоступным, — решением преподавателя по разделу «Введение» методических указаний; остальные оси при этом не меняются. Справочные разделы — в методических указаниях; задание ссылается на них по названию раздела.

Лабораторная работа 1. Стенд и сбор данных

Цель

Развернуть стенд курса, написать сборщик данных своего источника и запустить его службой, загрузить исторический срез, заполнить паспорт источника и начать каталог данных.

Исходные данные

- источник — **sensor.community, датчики качества воздуха**, описание — раздел «Источники данных» методических указаний;
- фильтр потока — **страны PL, CZ и AT**; значение `SOURCE_FILTER` — в листинге шага 3;
- период исторического среза — **одни сутки** до начала записи потока;
- предел объёма слоя сырых данных — `COLLECTOR_RAW_LIMIT_GB=20`;
- контракт сборщика — раздел «Контракт сборщика данных», форма паспорта — раздел «Источники данных», форма каталога — раздел «Каталог данных».

Предусловия

- рабочая машина отвечает требованиям раздела «Стенд курса»;
- у стенда есть доступ в интернет, в том числе к GitHub.

Задание

Стенд

1. По инструкции раздела «Стенд курса» создайте виртуальную машину или дистрибутив WSL2 и установите Docker Engine.
2. По той же инструкции клонируйте репозиторий стенда в `~/stand`: клон — рабочий репозиторий, `data/`, `.env` и `.venv/` в нём уже исключены.
3. Создайте `.env` из `.env.example` и задайте параметры варианта:

```
VARIANT=27
SOURCE=sensors
SOURCE_FILTER=PL,CZ,AT
COMPOSE_FILE=compose.yaml
COLLECTOR_MODE=record
```

4. По готовому файлу подготовьте каталоги данных и соберите оболочку службы сборщика: `docker compose run --rm stand-init`, затем `docker compose build collector`.

Сборщик

5. Изучите источник по разделу «Источники данных» и начните паспорт `SOURCE.md` по форме того же раздела: доступ, лицензия, лимиты, поля события, ключ и время события.
6. Напишите сборщик в `collector/src/main.py` по разделу «Контракт сборщика данных»: режим записи `record` с фильтром задания, ключом `event_key` и временем события `event_time` в UTC, сжатыми часовыми файлами, сохранением позиции для перезапуска, ротацией по пределу объёма, повтором с растущей паузой и журналом строками JSON. Адреса и параметры программа читает из переменных окружения.

   Поток — опрос раз в 5 минут адреса с фильтром по стране, по одному запросу на страну задания. Окна соседних опросов перекрываются: замер, пришедший второй раз, отбрасывается по ключу — id датчика и `timestamp`. История — архивные файлы датчиков, которые поток отдаёт для стран задания.

7. Добавьте режим загрузки истории `backfill`: срез за одни сутки до начала записи, период — переменными `COLLECTOR_HISTORY_FROM` и `COLLECTOR_HISTORY_TO`. По готовому файлу загрузите срез: `docker compose run --rm collector backfill`.
8. По готовому файлу запустите сборщик службой: `docker compose --profile lab1 up -d`. Проверьте журнал (`docker compose logs -f collector`) и появление файлов в `data/raw/stream`.
9. Перезагрузите машину стенда и убедитесь, что запись продолжилась без входа в систему. По готовому файлу перезапустите службу: `docker compose restart collector`. В сутках перезапуска нет повторяющихся ключей — команда печатает 0:

```
zcat data/raw/stream/dt=<дата>/hour=*/*.jsonl.gz \
  | jq -r .event_key | sort | uniq -d | wc -l
```

Паспорт, каталог, точка восстановления

10. Через сутки записи измерьте объёмы: событий в минуту до фильтра и после него (по журналу), байт в сутки до сжатия и после. Пересчитайте, сколько займут восемь недель потока вместе с историческим срезом, и сравните с пределом. Назовите ограничение, которое наступит первым при росте потока. Внесите числа, ограничение, глубину истории и найденные дефекты в паспорт.
11. Заведите каталог данных `CATALOG.md` по разделу «Каталог данных» и добавьте строки наборов `raw.stream` и `raw.history`.
12. Заведите журнал работ `NOTES.md`: раздел «ЛР1» с решениями по сборщику и замерами. Зафиксируйте результат коммитом.
13. После защиты создайте точку восстановления стенда и копию слоя сырых данных вне виртуальной машины по разделу «Стенд курса»; копия обновляется не реже раза в неделю. Обе проверяются на защите работы 3.

Дополнительное задание

Добавьте сборщику счётчики на порту 9101 по разделу «Контракт сборщика данных» и покажите рост счётчика ошибок: отключите сеть стенда на минуту и включите снова.

Приёмка

Основное задание — 8 баллов, дополнительное — 1 балл. Сроки считаются от занятия 1.

Основное задание:

- [ ] стенд запускается одной командой, сборщик работает службой и продолжает запись после перезапуска машины;
- [ ] слой сырых данных разложен по часу получения, файлы сжаты, каждое событие несёт `event_key` и `event_time` в UTC, событий вне фильтра задания нет;
- [ ] исторический срез за период задания лежит в префиксе `history`, отдельно от записи потока;
- [ ] сборщик соблюдает предел частоты источника, повторяет запрос с паузой и продолжает запись с сохранённой позиции; объём слоя в пределе;
- [ ] паспорт источника заполнен, его объёмы совпадают с пересчётом по файлам интервала наблюдения; названо ограничение, которое наступит первым при росте потока;
- [ ] в каталоге данных есть строки `raw.stream` и `raw.history`.

Дополнительное задание:

- [ ] счётчики сборщика отдаются на порту 9101, при отключении сети растёт счётчик ошибок.

Лабораторная работа 2. Форматы данных

Цель

Описать схему события средством варианта, привести сырые данные к Parquet с партиционированием по времени события и обосновать выбор формата замерами.

Исходные данные

- способ описания схемы — **Avro**;
- слой сырых данных и паспорт источника ЛР1;
- встраиваемый движок DuckDB — запуск и запросы по инструкции подраздела «Встраиваемый движок» раздела «Сервисы стенда».

Предусловия

- основное задание ЛР1 принято, сборщик пишет поток;
- на стенде заведено виртуальное окружение Python (`python3 -m venv .venv`).

Задание

Схема события

1. Задайте в `.env` значение `SERDE=avro`.
2. Опишите схему события своего источника в каталоге `schemas/` по полям паспорта: имена, типы, обязательность, смысл в комментарии. Ключ и время события — обязательные поля, время — в миллисекундах UTC.

   Файл схемы — `schemas/event.avsc`. Время события — `long` с логическим типом `timestamp-millis`. Кодирование и чтение — пакет `fastavro`.
3. Напишите программу проверки `schemas/check.py`: она читает события за час из слоя сырых данных, кодирует их по схеме в двоичный вид и читает обратно. События, не прошедшие кодирование, выводятся с причиной; расхождение исправляется в схеме, а дефект источника вносится в паспорт. Программа печатает размер часа в JSON и в двоичной кодировке.

Колоночный формат

4. По инструкции подраздела «Встраиваемый движок» разберите слой сырых данных в DuckDB: типы, которые движок вывел сам, число событий по часам, повторы по ключу. Отличия выведенных типов от схемы запишите в `NOTES.md`.
5. Напишите запрос `jobs/to_parquet.sql`: весь слой сырых данных — поток и исторический срез — в Parquet с типами по схеме события и партициями по дате события, каталог `data/duckdb/parquet/events`, ключ партиции — `event_date` (подраздел «Встраиваемый движок»). Повторный запуск заменяет файлы, а не добавляет копии. Запуск по инструкции: `docker compose run --rm -T duckdb < jobs/to_parquet.sql`.
6. Выберите запрос замеров к своим данным (например, число событий по часам за сутки в разрезе главного признака источника) и выполните его по инструкции на сырых файлах и на Parquet. Время — медиана трёх запусков с `.timer on`, объём — `du -sh` каталогов за один и тот же период. Числа внесите в `NOTES.md`.

Эволюция схемы

7. Добавьте в схему новое поле со значением по умолчанию — поле источника, которое версия 1 не хранила. Версии 1 и 2 схемы — разные коммиты.
8. Покажите программой проверки, что данные, закодированные версией 1, читаются версией 2, и назовите вид совместимости. Проверьте обратное направление и объясните результат.

9. Запросом версии 2 по инструкции запишите в Parquet только новые сутки: старые партиции остаются без поля. Прочитайте весь набор одним запросом с `union_by_name = true` — у старых файлов поле пусто.
10. Добавьте в каталог данных строку набора `parquet.events` и раздел «ЛР2» в `NOTES.md`.

Дополнительное задание

Сравните параметры записи Parquet на своих данных: кодеки `uncompressed`, `snappy`, `gzip`, `zstd` и размер группы строк — не меньше трёх значений параметра `ROW_GROUP_SIZE`. Для каждого сочетания — размер набора и время запроса замеров; число групп строк и размер каждой показывает функция `parquet_metadata()`. Выберите параметры для очищенного слоя ЛР3 и обоснуйте выбор замерами.

Приёмка

Основное задание — 10 баллов, дополнительное — 1 балл. Сроки считаются от занятия 2.

Основное задание:

- [ ] схема события описана средствами Avro в `schemas/`, версии 1 и 2 лежат в истории репозитория;
- [ ] сырые данные преобразованы в Parquet с типами по схеме и партициями по дате события, повторный запуск запроса не создаёт копий;
- [ ] приведены замеры объёма и времени запроса замеров на сырых файлах и на Parquet;
- [ ] данные, закодированные версией 1 схемы, читаются версией 2, вид совместимости назван;
- [ ] каталог данных дополнен строкой `parquet.events`.

Дополнительное задание:

- [ ] кодеки и размер группы строк сравнены на собственных данных, выбор обоснован замерами.

Лабораторная работа 3. Озеро данных

Цель

Построить озеро данных на объектном хранилище: слои сырых и очищенных данных, соглашение об именовании, партиционирование и сроки хранения.

Исходные данные

- объектное хранилище — **Garage**, подраздел «Объектное хранилище» раздела «Сервисы стенда»;
- слой сырых данных ЛР1, схема события и запрос преобразования ЛР2;
- параметры Parquet — из дополнительного задания ЛР2 или по умолчанию.

Предусловия

- основное задание ЛР2 принято;
- точка восстановления после ЛР1 создана, копия слоя сырых данных вне виртуальной машины обновлялась на этой неделе.

Задание

Хранилище и соглашения

1. Задайте в `.env` значение `S3=garage` и допишите в конец `COMPOSE_FILE` через двоеточие файл хранилища `compose.s3-garage.yaml`.
2. По готовому файлу поднимите профиль работы: `docker compose --profile lab3 up -d`. Bucket `lake` создаётся сам.

   Проверьте bucket командой `docker compose exec s3 /garage bucket info lake`.
3. Запишите в `CATALOG.md` соглашения озера по разделу «Каталог данных»: слои и их префиксы, шаблон ключа объекта, имена ключей партиций, правило имени файла, при котором повторная запись заменяет объект. Объясните раскладку свойствами объектного хранилища: неизменяемость объекта, отсутствие переименования, цена листинга.

Слой сырых данных в озере

4. Добавьте сборщику копирование закрытых файлов в озеро по разделу «Контракт сборщика данных»: префикс — переменная `COLLECTOR_LAKE_PREFIX`, раскладка потока и истории повторяет раскладку на диске, объект, который уже лежит в озере, повторно не загружается. Клиент — `boto3` в `collector/requirements.txt` (подраздел «Объектное хранилище»).
5. По готовому файлу пересоберите оболочку и перезапустите службу: `docker compose build collector`, `docker compose up -d collector`. При старте сборщик докопирует накопленное с ЛР1. Сверьте число и объём файлов на диске и в озере.

Слой очищенных данных

6. Сформулируйте правила очистки для своего источника: повторы по ключу, недопустимые и служебные записи, приведение типов по схеме ЛР2, время в UTC. Правила и найденные дефекты запишите в раздел «ЛР3» `NOTES.md`.

   Для sensor.community: повторы замера в перекрывающихся окнах, значения строками, физически невозможные значения.

7. Напишите запрос очистки `jobs/clean.sql` для DuckDB за сутки D, заданные первой строкой файла. Опоздавшие события суток D лежат в сырье суток D+1, поэтому запрос читает из озера сырьё суток D и D+1 — поток и исторический срез — и пишет в слой очищенных данных Parquet только партицию D по дате события `event_date`. Сутки D очищаются, когда в озере есть сырьё суток D+1; повторный запуск даёт тот же результат, а не копию. Пример — подраздел «Встраиваемый движок».
8. Очистите все накопленные сутки, включая исторический срез. Сверьте число событий в партиции очищенного слоя за одни сутки с числом событий в сырье этих суток и объясните разницу правилами очистки и опоздавшими событиями.
9. Выполните запрос замеров ЛР2 прямо по файлам очищенного слоя в хранилище: `read_parquet('s3://lake/clean/…', hive_partitioning = true)` с предикатом по партиции. По `EXPLAIN ANALYZE` покажите число прочитанных файлов с предикатом и без.

Сроки хранения

10. Назначьте срок хранения каждого слоя и запишите его в соглашения: слои на конец семестра вместе с образами сервисов помещаются на диск стенда (`du -sh data/*`, `df -h`).
11. Напишите программу `jobs/retention.py` на `boto3`: удаляет из слоя объекты партиций старше срока, режим `--dry-run` печатает, что будет удалено. Проверьте её на слое очищенных данных со сроком в двое суток в режиме `--dry-run`.
12. Обновите в каталоге строки `raw.stream` и `raw.history` — копия в озере — и добавьте строку `clean.events`; у `parquet.events` ЛР2 отметьте потребителей.

Дополнительное задание

Рассчитайте хранение платформы: объём каждого слоя сейчас, прогноз на конец семестра и при росте потока на три порядка. Предложите уровни хранения — какие данные и через какой срок уходят на более дешёвый уровень или удаляются — и оцените стоимость года хранения по ценам одного облачного объектного хранилища.

Приёмка

Основное задание — 12 баллов, дополнительное — 1 балл. Сроки считаются от занятия 3.

Основное задание:

- [ ] хранилище Garage работает, в bucket’е `lake` данные разложены по слоям сырых и очищенных данных;
- [ ] соглашение об именовании ключей и партиций записано в `CATALOG.md` и объяснено свойствами объектного хранилища, листинг bucket’а ему соответствует;
- [ ] очистка отделена от сбора: `jobs/clean.sql` читает сырьё из озера и пишет только партицию D, правила очистки записаны, повтор не создаёт копий;
- [ ] запрос к Parquet выполняется прямо из хранилища, предикат по партиции сокращает число прочитанных файлов;
- [ ] сроки хранения слоёв заданы и обоснованы объёмом, `jobs/retention.py` удаляет устаревшее; каталог данных дополнен;
- [ ] точка восстановления после ЛР1 создана и датирована, копия слоя сырых данных лежит вне виртуальной машины и не старше недели.

Дополнительное задание:

- [ ] приведены объём по слоям, прогноз на семестр и оценка при росте потока на три порядка, предложены уровни хранения.

Лабораторная работа 4. Таблицы Iceberg

Цель

Создать таблицу Iceberg поверх слоя очищенных данных, запрашивать её движком варианта и обслуживать: снимки, откат, компакция, эволюция схемы.

Исходные данные

- движок запросов — **Trino**, подраздел «Движок запросов» раздела «Сервисы стенда»;
- каталог таблиц — Lakekeeper по REST, склад `lake`, пространство имён `clean` (подраздел «Каталог таблиц»);
- инструмент обслуживания таблиц — Spark, подраздел «Пакетный движок и обслуживание таблиц»;
- файлы подключений — каталог `conf/` рабочего репозитория, учётные данные — подраздел «Адреса и учётные данные»;
- слой очищенных данных ЛР3 и схема события ЛР2.

Предусловия

- основное задание ЛР3 принято;
- очищенный слой содержит всю накопленную историю по суткам.

Задание

Таблица и запросы

1. Задайте в `.env` значение `ENGINE=trino` и допишите в конец `COMPOSE_FILE` через двоеточие файл движка `compose.engine-trino.yaml`.
2. По готовому файлу поднимите профиль работы: `docker compose --profile lab4 up -d`. Склад `lake` в каталоге таблиц создаётся сам.
3. Подключите DuckDB к каталогу файлом `data/duckdb/init.sql` по подразделу «Встраиваемый движок». Создайте пространство имён `lake.clean` и таблицу `lake.clean.events` со столбцами по схеме события и скрытым партиционированием по дню времени события.
4. Напишите запрос загрузки `jobs/iceberg_load.sql`: период из слоя очищенных данных — сутки или час, заданные переменными в начале файла, — вставляется в таблицу; повторная загрузка того же периода сначала удаляет его строки. Загрузите историю по суткам, а последние сутки — 24 вставками по часу, как приходили бы часовые файлы потока. Каждая загрузка даёт снимок и новый файл данных.
5. Подключитесь к таблице движком Trino. Выполните запрос замеров ЛР2 к таблице без предиката по времени события и с предикатом на одни сутки. Для каждого запуска запишите число прочитанных файлов и байт.

   Каталог `lake` описывается файлом `conf/trino/lake.properties` по подразделу «Движок запросов», после чего `engine` перезапускается. Числа даёт `EXPLAIN ANALYZE`: разбиения, строки и байты оператора `TableScan`.

Снимки и откат

6. По инструкции подраздела «Пакетный движок и обслуживание таблиц» напишите подключение Spark `conf/spark/spark-defaults.conf` и выведите снимки таблицы в Spark SQL. Запросы Spark держите в `jobs/maintenance.sql`.

7. Сделайте ошибочную запись: загрузите одни сутки повторно без удаления или вставьте строки с неверным временем события. Покажите число строк до и после ошибки запросом к прошлому снимку (`VERSION AS OF`) и откатите таблицу к снимку до ошибки процедурой `rollback_to_snapshot`.

Обслуживание

8. Запишите число файлов данных и их средний размер по партициям, затем выполните компакцию `rewrite_data_files` с параметрами по умолчанию и повторите замер. Объясните, почему компакция переписала только партицию последних суток.
9. Выполните устаревание снимков `expire_snapshots`, оставив последний. Запишите число снимков и число объектов таблицы в bucket'е до и после.
10. Добавьте в таблицу столбец поля, введённого версией 2 схемы в ЛР2. Покажите, что файлы данных не переписаны: список файлов до и после совпадает, у старых строк новое поле пусто.
11. Внесите числа в раздел «ЛР4» `NOTES.md` и добавьте в каталог строку `lake.clean.events`. Заведите в каталоге раздел «Связи сервисов» по разделу «Каталог данных»: связи сервисов профиля работы, Spark и DuckDB.

Дополнительное задание

Выполните две записи в таблицу одновременно из двух сеансов — например, обновление одних и тех же суток из Spark и из DuckDB. Покажите по снимкам и сообщениям об ошибке, как каталог разрешил конфликт коммитов, какая запись победила и почему. Сравните с двумя одновременными добавлениями строк в разные сутки. Таблицу после опыта верните откатом к снимку до него.

Приёмка

Основное задание — 15 баллов, дополнительное — 2 балла. Сроки считаются от занятия 4.

Основное задание:

- [ ] таблица `lake.clean.events` создана по схеме события с партиционированием по дню, зарегистрирована в каталоге и наполнена историей из слоя очищенных данных;
- [ ] движок Trino выполняет запрос к таблице, предикат по времени события сокращает число прочитанных файлов и байт, числа до и после приведены;
- [ ] выполнены запрос к прошлому снимку таблицы и откат ошибочной записи;
- [ ] выполнены компакция и устаревание снимков, числа файлов и снимков до и после приведены;
- [ ] столбец добавлен без перезаписи файлов данных; каталог данных дополнен строкой таблицы и разделом «Связи сервисов», подключения Spark и движка к каталогу таблиц лежат в `conf/` или заданы командой SQL.

Дополнительное задание:

- [ ] показаны две одновременные записи и разрешение конфликта снимков, объяснено, какая запись победила и почему.

Лабораторная работа 5. Хранилище и витрина

Цель

Спроектировать и наполнить витрину аналитического хранилища с фактами, измерениями и историей изменения, построить по ней дашборд и сравнить три пути к ответу на один вопрос.

Исходные данные

- аналитическое хранилище — ClickHouse, подраздел «Аналитическое хранилище» раздела «Сервисы стенда»;
- инструмент визуализации — **Apache Superset**, подраздел «Инструмент визуализации»: подключение и правила оформления графиков;
- слой очищенных данных ЛР3, таблица `lake.clean.events` ЛР4, движок Trino.

Предусловия

- основное задание ЛР4 принято;
- слой очищенных данных и таблица `lake.clean.events` содержат всю историю с начала записи, к защите — не меньше трёх недель (раздел «Порядок работ» методических указаний);
- если выполнялось дополнительное задание ЛР4, таблица `lake.clean.events` возвращена к снимку до опыта с конкурентной записью.

Задание

Слои хранилища

1. Задайте в `.env` значение `BI=superset` и допишите в конец `COMPOSE_FILE` через двоеточие файл инструмента визуализации `compose.bi-superset.yaml`.
2. По готовому файлу поднимите профиль работы: `docker compose --profile lab5 up -d`.
3. Заведите в ClickHouse слои приёма, ядра и витрин — базы `stg`, `core` и `mart` — и запишите назначение каждого слоя в раздел «ЛР5» `NOTES.md`.
4. Напишите загрузку `jobs/stg_load.sql`: сутки слоя очищенных данных читаются из объектного хранилища функцией `s3` и вставляются в `stg.events`; повторная загрузка тех же суток заменяет партицию, а не добавляет копию. Загрузите всю историю.

Витрина

5. Спроектируйте витрину по модели «звезда»: таблица фактов и два-три измерения. Назовите зернистость факта и ключи измерений, выберите порядок сортировки и партиционирование таблицы фактов и обоснуйте их запросами дашборда; решения запишите в `NOTES.md`.
6. У одного измерения ведите историю изменения атрибутов: версия строки со сроком действия. Измерение наполняется из `core`, где изменения атрибутов собраны по времени события. Если у кандидата за накопленную историю изменений нет, история ведётся по производному атрибуту — классу активности сущности за сутки по числу её событий.

   Кандидат — датчик: место установки и координаты меняются при переносе.
7. Напишите запросы `jobs/mart_load.sql`: `stg` → `core` → `mart`, повторяемые за те же сутки. Агрегат для дашборда — например, число событий в час по главному измерению — считает материализованное представление над таблицей фактов.
8. Выполните запрос «как выглядели данные на дату»: значение атрибута измерения на прошлую дату и на сегодня для одного и того же ключа.

Дашборд и три пути

9. Сформулируйте два-три вопроса к данным своего источника и постройте дашборд в Apache Superset: на каждый вопрос — график. Для каждого графика запишите вопрос, выбранный тип графика и почему он подходит.
10. Выберите один вопрос и получите ответ по всей истории тремя путями: по файлам озера (DuckDB над Parquet очищенного слоя), по таблице lakehouse (движок Trino) и по витрине (ClickHouse). Для каждого пути приведите медиану времени трёх запусков и прочитанный объём — по плану или профилю запроса, у ClickHouse — по строке `Processed` клиента; ответы совпадают или расхождение объяснено.
11. Добавьте в каталог строки таблиц `stg`, `core` и `mart` и связи хранилища и инструмента визуализации.

Дополнительное задание

Объясните, откуда берётся разница времени между тремя путями: сколько данных читает каждый путь, где лежит формат хранения, что делает планировщик запроса. Предскажите, как изменится время каждого пути при росте объёма данных на три порядка, и проверьте прогноз на наборе, в котором одни сутки повторены десять раз с новым ключом события. Набор лежит в отдельном префиксе озера, отдельной таблице lakehouse и отдельных таблицах витрины, рабочие наборы следующих работ не меняет и после замера удаляется.

Приёмка

Основное задание — 15 баллов, дополнительное — 2 балла. Сроки считаются от занятия 5.

Основное задание:

- [ ] хранилище разделено на слои приёма, ядра и витрин, назначение слоёв записано;
- [ ] витрина построена по модели «звезда»: зернистость факта и ключи измерений названы, порядок сортировки и партиционирование обоснованы запросами дашборда, агрегат дашборда считает материализованное представление;
- [ ] у одного измерения ведётся история изменения атрибутов, запрос «как выглядели данные на дату» выполняется;
- [ ] витрина наполняется из объектного хранилища файлами Parquet, повторная загрузка суток не создаёт копий; каталог данных дополнен;
- [ ] дашборд в Apache Superset отвечает на вопросы к данным, для каждого графика назван вопрос и обоснован тип графика;
- [ ] один вопрос решён по всей истории по файлам озера, по таблице lakehouse и по витрине; для каждого пути названа архитектура хранения, приведены время и прочитанный объём.

Дополнительное задание:

- [ ] разница времени трёх путей объяснена, прогноз при росте объёма на три порядка сделан и проверен.

Лабораторная работа 6. Поток событий

Цель

Передавать события источника через брокер по схеме из реестра, выгружать поток в озеро и наблюдать отставание потребителя.

Исходные данные

- брокер — **Redpanda**, реестр схем — по подразделу «Брокер и реестр схем» раздела «Сервисы стенда»;
- способ описания схемы — **Avro**, схема события ЛР2;
- режимы отправки и повтора — раздел «Контракт сборщика данных»;
- слой сырых данных в озере ЛР3.

Предусловия

- основное задание ЛР5 принято;
- сборщик копирует закрытые файлы в озеро.

Задание

Брокер и схема

1. Задайте в `.env` значение `BROKER=redpanda` и допишите в конец `COMPOSE_FILE` через двоеточие файл брокера `compose.broker-redpanda.yaml`.
2. По готовому файлу перейдите на профиль работы: движок, аналитическое хранилище и инструмент визуализации останавливаются, брокер и реестр запускаются.

```
docker compose --profile lab5 stop
docker compose --profile lab6 up -d
```

3. Выберите ключ сообщения и число партиций темы событий. Ключ определяет, какие события сохраняют порядок относительно друг друга; число партиций — сколько потребителей группы читают тему параллельно. Создайте тему командой подраздела «Брокер и реестр схем» и запишите обоснование в раздел «ЛР6» `NOTES.md`.
4. Зарегистрируйте схему события версии 2 из ЛР2 в реестре под именем `<тема>-value`. Проверьте несовместимое изменение запросом проверки подраздела «Брокер и реестр схем» — ответ `"is_compatible": false`, — затем попытайтесь его зарегистрировать — ответ 409. Назовите режим совместимости реестра.

   Несовместимое изменение — смена типа поля (`long` на `string`) или новое поле без значения по умолчанию.

Отправка и повтор

5. Добавьте сборщику режим отправки `produce`: каждое событие публикуется в тему с выбранным ключом, сериализатором реестра для Avro; запись в файлы продолжается, недоступность брокера запись не останавливает. Клиент — `confluent-kafka[avro]` строкой в `collector/requirements.txt`. Программа в контейнере сборщика видит только `collector/src/`.

   Схема для сериализатора берётся из реестра — последняя версия `<тема>-value`, зарегистрированная в шаге 4.
6. По готовому файлу пересоберите оболочку и переключите службу на отправку (раздел «Контракт сборщика данных», «Запуск»): `COLLECTOR_MODE=produce` в `.env`, затем `docker compose build collector` и `docker compose up -d collector`.

7. Добавьте режим повтора `replay`: записанный период из слоя сырых данных публикуется в отдельную тему с ускорением `COLLECTOR_REPLAY_SPEED`. Воспроизведите час записи и сравните число сообщений в теме с числом событий в файлах часа.

Выгрузка в озеро

8. Напишите потребитель `jobs/sink.py`: группа потребителей `sink` читает тему событий, разбирает сообщения по схеме из реестра и пишет файлы Parquet в слой сырых данных озера — набор `raw.topic`, раскладка по соглашениям ЛР3 с разбиением по часу. Смещение фиксируется после записи файла. Потребитель запускается на стенде в виртуальном окружении и обращается к брокеру, реестру и хранилищу по `localhost`.
9. Сверьте наборы `raw.topic` и `raw.stream` за общий период: запрос DuckDB находит ключи, которые есть только в одном из наборов. Расхождение объясните.
10. Остановите потребитель на несколько минут и покажите рост отставания группы командой подраздела «Брокер и реестр схем»; запустите снова и покажите, как отставание уходит в ноль.
11. Добавьте в каталог строки темы событий и набора `raw.topic` и связи брокера и реестра схем.

Дополнительное задание

Покажите распределение сообщений по партициям темы: число сообщений и объём на партицию. Найдите перекос по ключу — партицию, на которую приходится больше всего, — и объясните его следствие для порядка обработки и для параллельности потребителей. Предложите исправление, сохраняющее нужный порядок событий, и проверьте его на теме повтора.

Приёмка

Основное задание — 15 баллов, дополнительное — 2 балла. Сроки считаются от занятия 6.

Основное задание:

- [ ] сборщик публикует события в брокер, запись в файлы продолжается, потребитель `sink` читает сообщения и пишет файлы набора `raw.topic`;
- [ ] схема события зарегистрирована в реестре, несовместимое изменение не проходит проверку и отклоняется при регистрации ответом 409, режим совместимости назван;
- [ ] ключи `raw.topic` и `raw.stream` за общий период совпадают или расхождение объяснено; повтор записанного часа опубликован в отдельную тему, число сообщений равно числу событий часа;
- [ ] отставание группы `sink` растёт при остановке потребителя и уходит в ноль после возобновления;
- [ ] выбор числа партиций и ключа сообщения обоснован; каталог данных дополнен.

Дополнительное задание:

- [ ] показан перекос по ключу и его следствие для порядка обработки, исправление предложено и проверено.

Лабораторная работа 7. Пакетная обработка и конвейер

Цель

Считать агрегаты по истории пакетным заданием Spark, записывать их в таблицу lakehouse и в витрину и оформить счёт конвейером по расписанию с повторным счётом и проверками качества.

Исходные данные

- оркестратор — **Apache Airflow**, подраздел «Оркестратор» раздела «Сервисы стенда»: сессия Spark, DuckDB и клиент ClickHouse внутри оркестратора;
- пакетный движок — Spark, подраздел «Пакетный движок и обслуживание таблиц»;
- таблица `lake.clean.events` ЛР4, витрина ЛР5, запросы очистки и загрузки ЛР3 и ЛР4, движок Trino.

Предусловия

- основное задание ЛР6 принято;
- таблица `lake.clean.events` содержит всю историю с начала записи, к защите — не меньше четырёх недель (раздел «Порядок работ» методических указаний).

Задание

Пакетное задание

1. Задайте в `.env` значение `ORCHESTRATOR=airflow`, допишите в конец `COMPOSE_FILE` через двоеточие файл оркестратора `compose.orchestrator-airflow.yaml` и верните сборщик в режим записи: `COLLECTOR_MODE=record`.
2. По готовому файлу перейдите на профиль работы: брокер и реестр останавливаются, движок, аналитическое хранилище и оркестратор запускаются.

```
docker compose --profile lab6 stop
docker compose --profile lab7 up -d
```

3. Напишите задание PySpark `jobs/aggregate.py`: по таблице `lake.clean.events` за сутки, переданные аргументом, считает агрегаты по часу и главному измерению источника — число событий, число различных ключей сущности, сумма или среднее числового поля. Результат пишется в таблицу `lake.clean.events_hourly` с партицией по дню; повторный счёт тех же суток заменяет партицию, остальные сутки не трогает (способ замены — подраздел «Пакетный движок и обслуживание таблиц»).
4. Запустите задание по инструкции подраздела «Пакетный движок и обслуживание таблиц» с публикацией порта 4040. По интерфейсу задания и плану `explain("formatted")` найдите стадии и перемешивание: сколько стадий, где и по какому ключу данные перемешиваются, сколько данных читается и видно ли в плане отсечение партиций по суткам. Запишите разбор в раздел «ЛР7» `NOTES.md`.
5. Загрузите агрегаты суток в витрину: таблица `mart.events_hourly` наполняется из таблицы lakehouse через базу над каталогом (подраздел «Аналитическое хранилище»), повторная загрузка заменяет партицию суток. Сверьте итоги суток: в таблице lakehouse — запросом движка Trino, в витрине — запросом ClickHouse.

Конвейер

6. Оформите счёт суток конвейером в `dags/`: очистка суток слоя сырых данных, загрузка в `lake.clean.events`, агрегат, загрузка витрины, проверки качества. Шаги очистки и загрузки выполняет DuckDB внутри оркестратора, агрегат — Spark. Шаги выполняют те же файлы `jobs/`, что запускаются вручную; в оркестраторе это `/opt/jobs` (подраздел «Оркестратор»). Плановый запуск — раз в сутки в 01:30 UTC за прошедшие сутки D: к этому времени в озере есть сырьё суток D+1 (ЛР3, шаг 7).

   Конвейер — DAG на декораторах `@dag` и `@task` из `airflow.sdk` с расписанием `30 1 * * *`; обрабатываемые сутки — логическая дата запуска минус одни сутки, а не текущая дата.

7. Задайте проверки качества: свежесть — последнее событие суток не старше порога источника; обязательные поля не пусты; значения в допустимом диапазоне. Для каждой проверки решите, останавливает ли нарушение конвейер или помечает данные, и запишите решение.
8. Включите конвейер и не выключайте до запуска по расписанию: его предъявляет журнал запусков. Выполните повторный счёт за прошедшие трое суток средствами оркестратора (подраздел «Оркестратор»).
9. Запустите конвейер за те же сутки второй раз и покажите, что числа строк в `lake.clean.events_hourly` и в `mart.events_hourly` не изменились.
10. Нарушьте одну проверку — например, загрузите сутки с пустым обязательным полем — и покажите реакцию конвейера.
11. Добавьте в каталог строки `lake.clean.events_hourly` и `mart.events_hourly` и связи оркестратора.

Дополнительное задание

Найдите перекос ключей в соединении событий с агрегатом по сущности: ключ, на который приходится несоразмерно много строк. Соединение — сортировкой со слиянием, широковещание отключено: `spark.sql.autoBroadcastJoinThreshold=-1`. Покажите перекос в интерфейсе задания — время задач стадии соединения. Примените исправление: соль ключа или широковещательное соединение. Приведите время стадии и распределение задач до и после.

Приёмка

Основное задание — 15 баллов, дополнительное — 2 балла. Сроки считаются от занятия 7.

Основное задание:

- [ ] задание Spark считает агрегаты по истории и пишет результат в `lake.clean.events_hourly`, витрина `mart.events_hourly` загружена из неё, итоги суток в двух таблицах совпадают;
- [ ] план задания объяснён: число стадий, где и по какому ключу происходит перемешивание, объём читаемых данных; отсечение партиций по суткам видно в плане;
- [ ] в журнале запусков есть запуск конвейера по расписанию, повторный счёт за прошедший период выполняется;
- [ ] повторный запуск за те же сутки не создаёт дублей ни в таблице lakehouse, ни в витрине;
- [ ] проверки свежести, обязательных полей и диапазона заданы, нарушение останавливает конвейер или помечает данные и видно в журнале запусков; каталог данных дополнен.

Дополнительное задание:

- [ ] перекос ключей в соединении показан, исправление применено, замеры до и после приведены.

Лабораторная работа 8. Потоковая обработка

Цель

Считать агрегаты на потоке событий с окнами и watermark, обрабатывать поздние события, выводить результат на панель и сравнивать его с пакетным счётом.

Исходные данные

- потоковый движок — **Apache Flink**, подраздел «Потоковый движок» раздела «Сервисы стенда»;
- брокер Redpanda с темой событий ЛР6, способ описания схемы Avro;
- агрегаты ЛР7 в `lake.clean.events_hourly`, инструмент визуализации Apache Superset.

Предусловия

- основное задание ЛР7 принято;
- схема события зарегистрирована в реестре, сборщик умеет режим отправки (ЛР6).

Задание

Потоковое задание

1. Задайте в `.env` значение `STREAM=flink`, допишите в конец `COMPOSE_FILE` через двоеточие файл потокового движка `compose.stream-flink.yaml` и верните сборщику режим `COLLECTOR_MODE=produce`.
2. По готовому файлу перейдите на профиль работы: движок и оркестратор останавливаются, брокер, реестр, аналитическое хранилище, визуализация и потоковый движок запускаются.

```
docker compose --profile lab7 stop
docker compose --profile lab8 up -d
```

3. Измерьте задержку событий своего источника: разность `received_at` и `event_time` в слое сырых данных — медиана, 95-й и 99-й процентили за сутки. По этим числам и по частоте поступления выберите размер окна и допуск watermark и запишите обоснование в раздел «ЛР8» `NOTES.md`.
4. Напишите потоковое задание: читает тему событий, разбирает сообщения по схеме из реестра, считает в окнах по времени события те же показатели, что задание ЛР7, и пишет результат в таблицу `lake.clean.events_rt`. Checkpoint'ы хранятся на диске стенда, фиксация в таблицу — не чаще раза в минуту. Раз в сутки таблица обслуживается командами ЛР4: компакция и устаревание снимков (подраздел «Потоковый движок»).

   Задание — файл Flink SQL `jobs/stream.sql`: каталог Iceberg, таблица источника с форматом `avro-confluent` и объявлением `WATERMARK`, окно — табличная функция `TUMBLE` или `HOP`. Запуск — SQL-клиентом в контейнере `stream`.
5. Сделайте результат витриной: в ClickHouse представление `mart.events_rt` читает `lake.clean.events_rt` через базу над каталогом (подраздел «Аналитическое хранилище»). Добавьте в дашборд ЛР5 панель текущих значений по этому представлению с обновлением раз в минуту.

Поздние события и сверка

6. Отправьте в тему событие, время которого старше закрытого окна: копию реального события с временем на два окна раньше watermark, с тем же сериализатором реестра, что у сборщика. Покажите, что случилось с ним в результате, и объясните по допуску watermark.

7. Сравните результат потока с пакетным счётом ЛР7 за одни и те же сутки: по часам и по главному измерению. Объясните расхождение: поздние события, повторы по ключу, границы окон, фильтр очистки; число различных значений за сутки не складывается из чисел по окнам.
8. Добавьте в каталог строки `lake.clean.events_rt` и `mart.events_rt` — набор реального времени — с ожидаемой свежестью и связи потокового движка.

Дополнительное задание

Перезапустите потоковое задание вместе со стендом: остановите профиль работы командой `docker compose --profile lab8 down` и поднимите снова. Восстановите задание с последнего checkpoint'а (подраздел «Потоковый движок») и покажите, что счёт продолжается без обнуления накопленных значений: окна, открытые до остановки, закрываются с полными итогами, дублей в таблице нет.

Приёмка

Основное задание — 10 баллов, дополнительное — 1 балл. Сроки считаются от занятия 8.

Основное задание:

- [ ] потоковое задание читает тему событий и пишет результат в `lake.clean.events_rt`, витрина `mart.events_rt` отдаёт его панели, панель показывает текущие значения;
- [ ] окно и watermark заданы, их размеры обоснованы измеренной задержкой событий источника;
- [ ] событие, пришедшее позже своего окна, отправлено в тему, его судьба в результате объяснена;
- [ ] результат потока сравнён с пакетным счётом ЛР7 за те же сутки, расхождение объяснено;
- [ ] каталог данных дополнен набором реального времени и связями потокового движка.

Дополнительное задание:

- [ ] после перезапуска стенда задание восстанавливается с checkpoint'а, окна вокруг остановки посчитаны полностью, дублей в таблице нет.

Лабораторная работа 9. Исследование данных

Цель

Ответить на собственный вопрос к данным всеми тремя архитектурами платформы, сравнить пути к ответу и представить ответ дашбордом. Работа необязательна.

Исходные данные

- платформа ЛР1–ЛР8: файлы озера, таблицы lakehouse, витрина;
- движок Trino, инструмент визуализации Apache Superset;
- правила оформления графиков — подраздел «Инструмент визуализации» раздела «Сервисы стенда».

Предусловия

- основные задания ЛР1–ЛР8 приняты;
- история источника в платформе — вся с начала записи, к защите — не меньше шести недель (раздел «Порядок работ» методических указаний).

Задание

Всё задание работы — дополнительное.

Вопрос и три пути

1. Верните сборщик в режим записи (`COLLECTOR_MODE=record`) и по готовому файлу перейдите на профиль работы: брокер, реестр и потоковый движок останавливаются.

```
docker compose --profile lab8 stop
docker compose --profile lab9 up -d
```

2. Сформулируйте вопрос к данным своего источника, на который не отвечает дашборд ЛР5 и который требует истории не меньше четырёх недель: сравнение периодов, распределение, связь двух признаков. Запишите вопрос и ожидаемую форму ответа в раздел «ЛР9» `NOTES.md`.
3. Получите ответ по всей истории тремя путями: по файлам очищенного слоя в озере (DuckDB), по таблицам lakehouse (движок Trino) и по витрине (ClickHouse; при необходимости — новая таблица витрины). Для каждого пути приведите медиану времени трёх запусков и прочитанный объём (как в ЛР5); ответы совпадают или расхождение объяснено.
4. Для каждого пути запишите, для какой задачи он уместен: разовое исследование, регулярный отчёт, дашборд на много пользователей.

Дашборд

5. Постройте в Apache Superset дашборд, отвечающий на вопрос. Для каждого графика запишите, какую часть вопроса он показывает, почему выбран этот тип графика и какие правила оформления применены.

Расширение платформы

6. Выполните одно расширение из списка и опишите его в каталоге данных:
   - второй источник курса со своим паспортом, слоем сырых данных и соединением с основным источником в витрине;
   - эволюция партиционирования `lake.clean.events` — например, по часу для новых данных — с замером прочитанных файлов до и после;
   - компакция `lake.clean.events` с сортировкой по второму столбцу предиката и замер отсечения файлов по нему;
   - история нарушений проверок качества ЛР7 отдельным набором и панель по ней;
   - материализованное представление ClickHouse, считающее агрегат ЛР7 при вставке фактов, и сравнение свежести с пакетной загрузкой.

Приёмка

Работа необязательна: 12 баллов, основного задания нет. Работа принимается целиком и только на девятом занятии.

Дополнительное задание:

- [ ] вопрос к данным сформулирован, ответ получен тремя путями: по файлам озера, по таблицам lakehouse, по витрине;
- [ ] время и прочитанный объём каждого пути приведены, названо, для какой задачи какой путь уместен;
- [ ] дашборд в Apache Superset отвечает на вопрос, выбор графиков обоснован;
- [ ] одно расширение из списка выполнено и описано в каталоге данных.